Assignment: Point-to-Point and Collective Communications in MPI/OpenMP

Section : A

Problem_1: Divide a large integer array among MPI processes. Each process exchanges its boundary elements with neighboring processes using point-to-point communication and computes a local result.

Sample Input

Number of processes: 4
N = 12
Array: 10 20 30 40 50 60 70 80 90 100 110 120
Distribution:

P0: 10 20 30; P1: 40 50 60; P2: 70 80 90; P3: 100 110 120

Sample Output

P0: Right boundary sent = 30
P1: Left boundary received = 30
P1: Right boundary sent = 60
P2: Left boundary received = 60
P2: Right boundary sent = 90
P3: Left boundary received = 90
Boundary exchange completed successfully.

Problem_2: The root process distributes different rows of a matrix to worker processes using MPI_Send. Workers calculate the sum of their assigned rows and return the results using MPI_Recv.

Sample Input

Number of processes: 4

Matrix:

1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

Each process receives one row.

Sample Output

P0: Row = 1 2 3 4 Sum = 10
P1: Row = 5 6 7 8 Sum = 26
P2: Row = 9 10 11 12 Sum = 42
P3: Row = 13 14 15 16 Sum = 58
Total matrix sum = 136

Problem_3: Implement a ring topology in which every MPI process sends a message to its next process and receives a message from its previous process. Measure the communication time for different message sizes.

Sample Input

Number of processes = 4
Initial value at P0 = 100
Communication: P0 → P1 → P2 → P3 → P0

Sample Output

P0 sends 100 to P1
P1 receives 100 from P0
P1 sends 100 to P2
P2 receives 100 from P1
P2 sends 100 to P3
P3 receives 100 from P2
P3 sends 100 to P0
P0 receives 100 from P3
Ring communication completed.

Problem_4: Implement a ping-pong program between two MPI processes using MPI_Send and MPI_Recv. Test message sizes from a few bytes to several megabytes and analyze communication latency and bandwidth.

Sample Input

Number of processes: 2
Number of iterations = 5
Message size = 1024 bytes

Sample Output

Message size : 1024 bytes
Iterations : 5
Total round trips : 5
Average round-trip : 0.000018 seconds
Average one-way : 0.000009 seconds
Repeat the experiment for 64 bytes, 256 bytes, 1 KB, 4 KB, 16 KB, 64 KB, and 1 MB.

Problem_5: Distribute matrix rows from the root process to other processes using MPI_Send. Each process performs its portion of matrix multiplication and sends the result back to the root.

Sample Input

Number of processes: 2
Matrix A:
1 2
3 4
Matrix B:
5 6
7 8

Sample Output

Matrix C = A × B
19 22
43 50

C[0][0] = 1×5 + 2×7 = 19
C[0][1] = 1×6 + 2×8 = 22
C[1][0] = 3×5 + 4×7 = 43
C[1][1] = 3×6 + 4×8 = 50

Problem_6: Divide a one-dimensional array among MPI processes. Exchange halo/boundary values between neighboring processes using MPI_Send and MPI_Recv, then perform a stencil computation.

Sample Input

Number of processes: 4

Array: 10 20 30 40 50 60 70 80
Radius = 1
Distribution:
P0: 10 20
P1: 30 40
P2: 50 60
P3: 70 80
Stencil: out[i] = left[i] + current[i] + right[i]

Sample Output

After halo exchange:
P0: [10 20] + right halo 30
P1: left halo 20, [30 40], right halo 50
P2: left halo 40, [50 60], right halo 70
P3: left halo 60, [70 80]
Stencil result (using the stated simple boundary convention):
30 60 90 120 150 180 210 230

Problem_7: Implement a distributed vector computation using MPI_Isend and MPI_Irecv. Compare its execution time with the equivalent blocking MPI_Send/MPI_Recv implementation.

Sample Input

Number of processes: 2
Array size = 8
P0: 10 20 30 40 50 60 70 80
P1: 1 2 3 4 5 6 7 8
Each process exchanges its array using MPI_Isend and MPI_Irecv.

Sample Output

P0 received: 1 2 3 4 5 6 7 8
P1 received: 10 20 30 40 50 60 70 80
Nonblocking communication completed.

Sample performance:

Blocking communication time = 0.000021 s
Nonblocking communication time = 0.000015 s

Problem_8: Create a two-process communication program that can cause deadlock when blocking sends are used. Modify the program to avoid deadlock using appropriate MPI communication techniques.

Sample Input

Number of processes: 2
P0 message = 100
P1 message = 200

Sample Output

Unsafe version:

P0: Waiting to send 100
P1: Waiting to send 200
Potential deadlock detected.

Corrected version:

P0 sent 100
P1 received 100
P1 sent 200
P0 received 200
Communication completed without deadlock.

Problem_9: Divide an array among MPI processes. Each process sorts its local portion and exchanges data with neighboring processes to produce a globally sorted array.

Sample Input

Number of processes: 4
Array: 45 12 78 3 56 23 89 10
Distribution:
P0: 45 12
P1: 78 3
P2: 56 23
P3: 89 10

Sample Output

Local sorting:
P0: 12 45
P1: 3 78
P2: 23 56
P3: 10 89

After data exchange and merge/exchange steps:
Sorted array: 3 10 12 23 45 56 78 89

Problem_10: Design a producer–consumer system where one or more MPI processes generate data and another process receives and processes the data using point-to-point communication.

Sample Input

Number of processes: 3
P0 = Producer
P1 = Consumer
P2 = Consumer
Number of items = 8
Items: 10 20 30 40 50 60 70 80

Sample Output

P0 produced: 10
P1 consumed: 10
P0 produced: 20
P2 consumed: 20
P0 produced: 30
P1 consumed: 30
P0 produced: 40
P2 consumed: 40
...
All 8 items processed successfully.

Section: B

Problem_11: The root process reads configuration parameters and broadcasts them to all MPI processes. Each process uses the received parameters to perform a computation.

Sample Input

Number of processes: 4
Root process reads:
Number of iterations = 1000
Threshold = 50
Block size = 256

Sample Output

P0 received: Iterations = 1000, Threshold = 50, Block size = 256
P1 received: Iterations = 1000, Threshold = 50, Block size = 256
P2 received: Iterations = 1000, Threshold = 50, Block size = 256
P3 received: Iterations = 1000, Threshold = 50, Block size = 256

Problem_12: Use MPI_Scatter to distribute a large vector among processes. Each process calculates the square or cube of its elements, and MPI_Gather collects the results at the root process.

Sample Input

Number of processes: 4
Vector: 1 2 3 4 5 6 7 8

After MPI_Scatter:

P0: 1 2
P1: 3 4
P2: 5 6
P3: 7 8

Operation: y[i] = x[i]^2

Sample Output

P0: 1 4
P1: 9 16
P2: 25 36
P3: 49 64

Gathered result: 1 4 9 16 25 36 49 64

Problem_13: Distribute matrix rows among processes when the number of rows is not evenly divisible by the number of processes. Collect the processed rows using MPI_Gatherv.

Sample Input

Number of processes: 3

Matrix:
1 2 3
4 5 6
7 8 9
10 11 12
13 14 15

Five rows are distributed as:

P0 → 2 rows
P1 → 2 rows

P2 → 1 row

Operation: multiply every element by 2.

Sample Output

P0:
2 4 6
8 10 12
P1:
14 16 18
20 22 24
P2:
26 28 30
Final matrix:
2 4 6
8 10 12
14 16 18
20 22 24
26 28 30

Problem_14: Each process generates a set of numerical values. Use MPI_Reduce to calculate the global sum, maximum, minimum, and average.

Sample Input

Number of processes: 4
P0: 10 20 30
P1: 40 50 60
P2: 70 80 90
P3: 100 110 120

Sample Output

Local sums:
P0 = 60
P1 = 150
P2 = 240
P3 = 330
Global Sum = 780
Global Maximum = 120
Global Minimum = 10
Average = 65.00

Problem_15: Implement the trapezoidal rule for numerical integration. Each MPI process calculates a portion of the integral, and MPI_Reduce combines the partial results.

Sample Input

Number of processes: 4
Function: $f(x) = x^2$
Lower limit = 0
Upper limit = 1
Number of intervals = 1000
Use the trapezoidal rule.

Sample Output

Numerical integral ≈ 0.333333
Exact integral = 0.333333
Absolute error ≈ 0.000000

Students should report partial integrals and repeat the experiment with different numbers of intervals.

Problem_16: Each MPI process stores a local value. Use MPI_Scan to calculate the prefix sum across all processes and compare it with a manually implemented point-to-point solution.

Sample Input

Number of processes: 4
P0 = 10
P1 = 20
P2 = 30
P3 = 40

Sample Output

P0: Local value = 10, Prefix sum = 10
P1: Local value = 20, Prefix sum = 30
P2: Local value = 30, Prefix sum = 60
P3: Local value = 40, Prefix sum = 100

Prefix result: 10 30 60 100

Problem_17: Each MPI process generates data intended for every other process. Implement the communication using MPI_Alltoall and analyze its communication cost as the number of processes increases.

Sample Input

Number of processes: 4
P0: 0 1 2 3
P1: 10 11 12 13
P2: 20 21 22 23
P3: 30 31 32 33
Each process sends one value to every process.

Sample Output

After MPI_Alltoall:
P0 received: 0 10 20 30
P1 received: 1 11 21 31
P2 received: 2 12 22 32
P3 received: 3 13 23 33

Students should measure communication time for P = 2, 4, 8, 16 and different message sizes.

Problem_18: Divide a large dataset among MPI processes. Each process generates a local histogram, and MPI_Allreduce combines all local histograms into a global histogram.

Sample Input

Number of processes: 4
Values range from 0 to 9.
P0: 1 2 2 3
P1: 2 3 4 4
P2: 1 4 5 5
P3: 0 2 5 9

Sample Output

Final histogram after MPI_Allreduce:

Value	Frequency
0	1
1	2
2	4
3	2
4	3
5	3

6	0
7	0
8	0
9	1

Total number of elements = 20